



Formularios en HTML5

Universidad de Sevilla

**Departamento de Lenguajes y Sistemas
Informáticos**

**Carlos Arévalo, Daniel Ayala, José Calderón,
Margarita Cruz, Inma Hernández, David Ruiz**

UNIVERSIDAD DE SEVILLA

Objetivos



Contenidos

Introducción

Elementos de formulario

Validación de formularios

Buenas prácticas



Contenidos

Introducción

Elementos de formulario

Validación de formularios

Buenas prácticas



¿Qué es un formulario?

SOLICITUD de INSCRIPCIÓN
ACADEMIA VALENCIANA de GENEALOGÍA y HERÁLDICA

PRIMER APELLIDO _____

SEGUNDO APELLIDO _____

NOMBRE _____ N.I.F. _____

DOMICILIO _____ TELEFONO _____

CÓDIGO POSTAL _____ LOCALIDAD _____ PROVINCIA _____

e-mail: _____ @ _____

Fecha _____

Firma _____

CUOTAS:

Derechos de inscripción	75 €
Cuota anual	60 €

FORMA DE PAGO: C.C.C.: _____

E S ____/____/____

DOCUMENTOS a PRESENTAR:

- Curriculo
- Fotografía tamaño carnet, en su reverso escribir:
Nombre, Apellidos y D.N.I.

Enviar a: **ACADEMIA VALENCIANA de GENEALOGÍA y HERÁLDICA**
Apartado de Correos 1834 46080 VALENCIA

Es un documento, físico o digital, que permite a un usuario introducir datos (nombres, apellidos, dirección, fecha, etc.) en las zonas correspondientes, para ser almacenados y procesados posteriormente.

¿Qué es un formulario HTML?

```
formulario.html > html > body > div > input
1 <html>
2   <head>
3
4   </head>
5   <body>
6     <div>
7       <h1>SOLICITUD DE INSCRIPCION</h1>
8       <h2>ACADEMIA VALENCIANA DE GENEALOGÍA Y HERÁLDICA</h2>
9
10      Primer apellido <input type="text" name="apellido1" id="apellido1" required><br>
11      Segundo apellido <input type="text" name="apellido2" id="apellido2" required><br>
12      Nombre <input type="text" name="nombre" id="nombre" required><br>
13      NIF <input type="text" name="nif" id="nif" required length="9"><br>
14      Domicilio <input type="text" name="domicilio" id="domicilio" required><br>
15      Teléfono <input type="text" name="telefono" id="telefono" required length="9"><br>
16      Código Postal <input type="text" name="cp" id="cp" required length="5">
17      Localidad <input type="text" name="localidad" id="localidad" required><br>
18      Provincia <input type="text" name="provincia" id="provincia" required><br>
19      e-mail <input type="text" name="email" id="email" required><br>
20      Fecha <input type="date" name="fecha" id="fecha" required><br>
21      Fotografía <input type="file" name="foto" id="foto" required><br>
22      FORMA DE PAGO C.C.C ES<input type="text" length="2" size="2"> -
23        <input type="text" length="4" size="4"> -
24        <input type="text" length="4" size="4"> -
25        <input type="text" length="2" size="2"> -
26
27    </div>
28  </body>
29
30</html>
```

SOLICITUD DE INSCRIPCION

ACADEMIA VALENCIANA DE GENEALOGÍA Y HERÁLDICA

Primer apellido

Segundo apellido

Nombre

NIF

Domicilio

Teléfono

Código Postal Localidad

Provincia

e-mail

Fecha

Fotografía Ningún archivo seleccionado

FORMA DE PAGO C.C.C ES - - - -

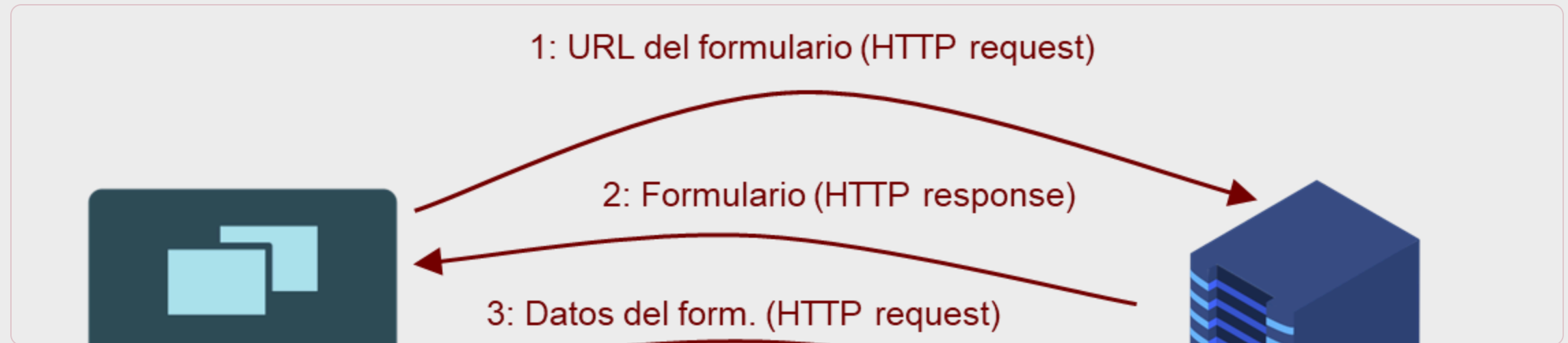
Es la implementación en HTML de un formulario digital para enviar datos a un servidor web, para su almacenado o procesamiento

Conceptos básicos

Los formularios permiten enviar información al servidor web mediante una interfaz de usuario muy básica.

El procesamiento de un formulario se realiza por parte del backend que se está ejecutando en el servidor (Django, Spring, Silence...)

Se pueden validar datos en el cliente antes de mandarlos al servidor mediante el uso de JavaScript.



Formularios en HTML5

Documento con formularios x +

← → ↻ ⓘ Archivo | C:/Users/inmahe... ☆ Incógnito ⋮

Nuevos tipos de input

*Los campos obligatorios están marcados con **

Nuevos tipos

NIF

email

URL

Search

Tel

Number

Range

Color

Tipos fecha/hora

Datetime-local

Date

Time

Month

Week

Tipos de select

Desplegable

Contenidos

Introducción

Elementos de formulario

Validación de formularios

Buenas prácticas



Elemento form

```
<form id="identificador único para procesado en cliente"
      action="URL de CGI, ASP, PHP, JSP, ..."
      method="get|post"
      enctype="multipart/form-data|text/plain"
      target="frame o ventana para la respuesta">

  <input .../>

  <select ...>
    <option>...</option>
    ...
  </select>

  <textarea>...</textarea>

  <button ...>...</button>

  <fieldset ...>...</fieldset>

  <label ...>...</label>
</form>
```

Atributos de `<form>`

- `id` : identifica al formulario para poder acceder a él desde código `JavaScript` de cliente.
- `action` : dirección que procesará los datos del formulario. En `HTML5` , si no se especifica, se asume que es la página actual. Puede ser también `mailto:nombre@direccion` si se quieren enviar los datos por correo electrónico.
- `enctype` : tipo de codificación de los datos. Si se van a enviar ficheros adjuntos debe ser `multipart/form-data` o `text/plain` si se va a enviar por correo. El valor por defecto es `application/x-www-form-urlencoded` , que convierte espacios en `+` , caracteres con código mayor de `127` en `%HH` (hexadecimal), saltos de línea en `%0D%0A` y `+` en `%2B` .
- `target` : nombre de marco o de ventana en el que se mostrarán los resultados del procesamiento del formulario (por defecto `_self` , la ventana actual).
- `novalidate` : indica al navegador que no se desea validar el formulario (en cliente)
- `autocomplete` : indica al navegador si debe ofrecer o no autocompletar el formulario. Por defecto es `on` .

Atributo method

Especifica **cómo se van a enviar los datos del formulario**. Puede tomar dos valores: `POST` y `GET`. Por defecto el valor es `GET`.

Método `GET`

- Los datos se añaden a la URL especificada en `action` después del carácter `?` en parejas `nombre=valor` separadas por `&`, por ejemplo `http://www.ejemplo.com/x.asp?nombre=Pepe&apellido=P%E9rez`

Método `POST`

- Primero se establece una conexión con la URL especificada en `action` y a continuación se envían los datos.
- Si se quieren enviar archivos adjuntos al servidor es la única opción.

GET vs POST

Para formularios con **pocos datos**, **GET** es **ligeramente más eficiente**. Para formularios con muchos datos, es mejor **POST**. Algunos servidores tienen un límite en la longitud de las URL.

Con **GET**, los valores de los datos se ven directamente en la URL. Con **POST** van “ocultos” en el cuerpo de la petición **HTTP**.

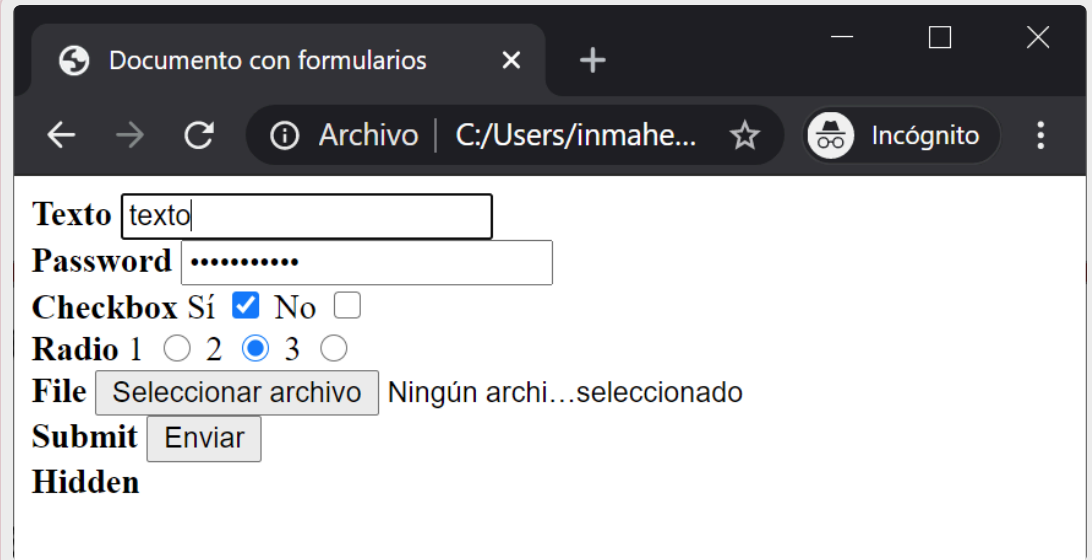
POST **no es más seguro que** **GET**, ya que si se intercepta la petición los valores enviados se pueden leer igualmente en texto plano. Si se usa **HTTPS** ambos son igualmente seguros.

Algunos **servidores pueden negarse** a procesar formularios enviados mediante **GET**.

Se aconseja usar **POST** para una mejor experiencia del usuario, ya que puede confundirse si ve los valores enviados en la URL.

Elemento input

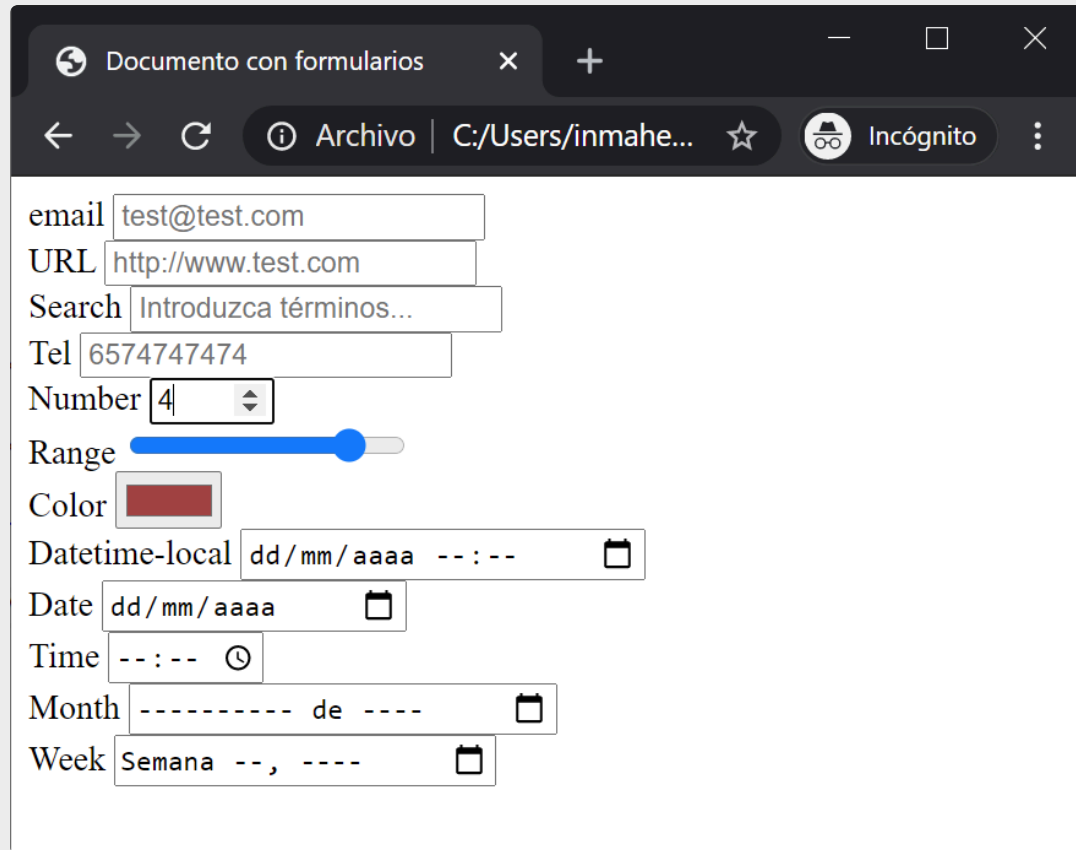
```
<form>
  <b>Texto</b> <input type="text"/><br/>
  <b>Password</b> <input type="password"/><br/>
  <b>Checkbox</b>
  Si <input type="checkbox" value="si"/>
  No <input type="checkbox" value="no"/><br/>
  <b>Radio</b>
  1 <input type="radio" value="1"/>
  2 <input type="radio" value="2"/>
  3 <input type="radio" value="3"/><br/>
  <b>File</b> <input type="file"/><br/>
  <b>Submit</b> <input type="submit"/><br/>
  <b>Hidden</b> <input type="hidden"/>
</form>
```



A screenshot of a web browser window displaying the rendered HTML form. The browser's address bar shows the file path C:/Users/inmahe... and the page title is 'Documento con formularios'. The form contains the following elements: a text input field with the value 'texto'; a password input field with masked characters '.....'; a checkbox section with 'Sí' checked and 'No' unchecked; a radio button section with '1' selected and '2' and '3' unselected; a file input field with the placeholder text 'Seleccionar archivo' and the status 'Ningún archi...seleccionado'; a submit button labeled 'Enviar'; and a hidden input field.

Elemento input (II)

```
<form>
  email <input type="email"
               name="email"
               placeholder="test@test.com"/>
  URL <input type="url"
            name="url"
            placeholder="http://www.test.com"/>
  Search <input type="search"
               name="search"
               placeholder="Introduzca términos..."/>
  Tel <input type="tel"
            name="telefono"
            placeholder="6574747474"/>
  Number <input type="number"
               name="number"
               min="1" max="5"/>
  Range <input type="range"
              name="range"
              min="0" max="50"/>
  Color <input type="color" name="color"/>
  Datetime-local <input type="datetime-local"
                       name="dl"/>
  Date <input type="date" name="date"/>
  Time <input type="time" name="time"/>
  Month <input type="month" name="month"/>
  Week <input type="week" name="week"/>
</form>
```



A screenshot of a web browser window titled "Documento con formularios". The browser shows a form with the following fields and their rendered states:

- email**: A text input containing "test@test.com".
- URL**: A text input containing "http://www.test.com".
- Search**: A text input containing the placeholder "Introduzca términos...".
- Tel**: A text input containing "6574747474".
- Number**: A numeric input containing the value "4".
- Range**: A range slider with a blue track and a blue handle.
- Color**: A color input showing a red color swatch.
- Datetime-local**: A datetime input showing the format "dd/mm/aaaa --:--" with a calendar icon.
- Date**: A date input showing the format "dd/mm/aaaa" with a calendar icon.
- Time**: A time input showing the format "--:--" with a clock icon.
- Month**: A month input showing the format "----- de ----" with a calendar icon.
- Week**: A week input showing the format "Semana --, ----" with a calendar icon.

Elemento input (III)

```
<input
  name="nombre para procesamiento en servidor"
  id="ident. unico para procesamiento en cliente"
  type="button|checkbox|file|hidden|image|
        password|radio|reset|submit|text|
        email|url|search|tel|number|range|color|
        datetime-local|date|month|week|time"
  value="valor por defecto o texto de boton"
  size="tamano del campo de texto"
  maxlength="tamano maximo del campo"
  checked
  title="texto para el tooltip"
  disabled readonly autofocus required novalidate
  pattern="expresionRegular"
  placeholder="texto para guiar al usuario"
/>
```

Atributos de `<input>`

`name` : indica el nombre del valor que se le envía al servidor (`name=valor`). Si no se especifica `name` , el valor del control no se envía.

`id` : identificador que se usa para acceder al control desde código `JavaScript` o aplicar estilo `css` .

`type` : indica el tipo de control.

`value` : indica el valor que se envía al servidor en el par `nombre=valor` . En algunos casos, `value` se usa como el texto de un botón.

`disabled` : si está presente, el control está deshabilitado.

`readonly` : si está presente, el control es de sólo lectura.

`size` : indica el tamaño en caracteres de los controles de entrada de texto.

Atributos de `<input>` (II)

`maxLength` : indica el número máximo de caracteres que aceptará un control de entrada de texto.

`checked` : si está presente, las casillas de verificación (`checkbox`) y los botones radio (`radio`) aparecen seleccionados.

`title` : añade información al control, normalmente visualizada como un `tooltip` .

`autofocus` : coloca el foco en el elemento

`required` : califica el campo como obligatorio

`novalidate` : el navegador no validará el campo

`pattern` : expresión regular que debe cumplir el contenido ingresado por el usuario

`placeholder` : texto que aparece para indicar al usuario lo que se espera que ingrese en ese campo

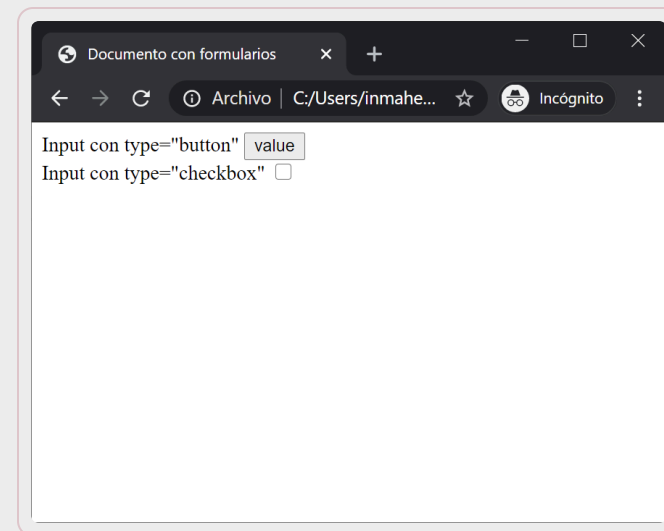
Botones / checkboxes

```
<input ... type="button"/>
```

- Genera un botón con el texto que indique el atributo value. Sólo tiene sentido si se gestiona el evento de pulsación en JavaScript.
- El elemento `<button>` lo ha convertido en obsoleto.

```
<input ... type="checkbox"/>
```

- Genera una casilla de comprobación que sólo se envía al servidor si está pulsada. El atributo checked hace que aparezca como seleccionada.
- El atributo value no genera texto, y si no se especifica el valor por defecto es on.



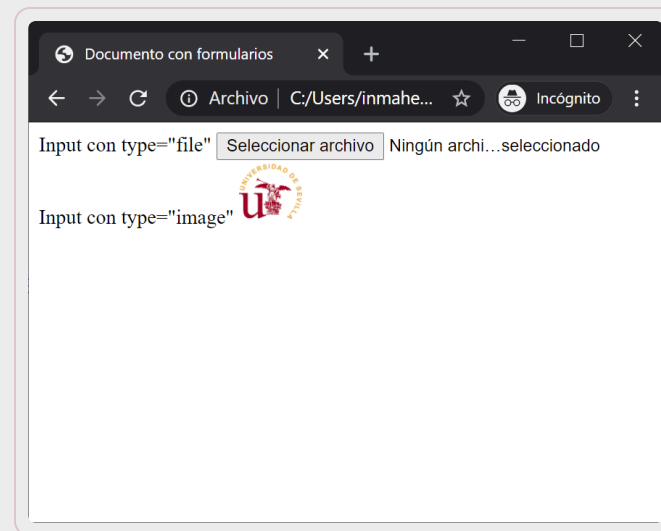
Ficheros / imágenes de control

```
<input ... type="file"/>
```

- Genera un control para enviar (upload) archivos al servidor. Se debe usar con formularios que tengan `enctype="multipart/form-data"` y `method="post"` ; si no, envía sólo el nombre del archivo.

```
<input ... type="image" src="URL imagen" />
```

- Genera una imagen que al pulsarse hace que se envíe el formulario, incluyendo las coordenadas dónde se pulsó el ratón como nombre.x y nombre.y.
- Es la forma de implementar los mapas en el servidor.



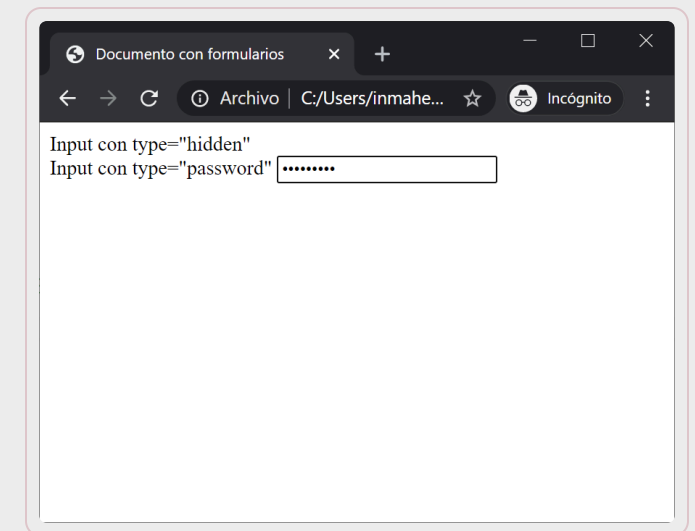
Campos ocultos / Contraseñas

```
<input ... type="hidden"/>
```

- Genera un campo oculto del formulario (no se visualiza) que siempre se envía al servidor, por lo que name y value son obligatorios.
- Es útil para la programación de aplicaciones web (como las cookies).

```
<input ... type="password"/>
```

- Genera un campo de texto en el que no se muestra el texto escrito.
- No implica que el envío de los datos esté encriptado, el contenido se envía en texto plano igualmente. Es un efecto únicamente visual.



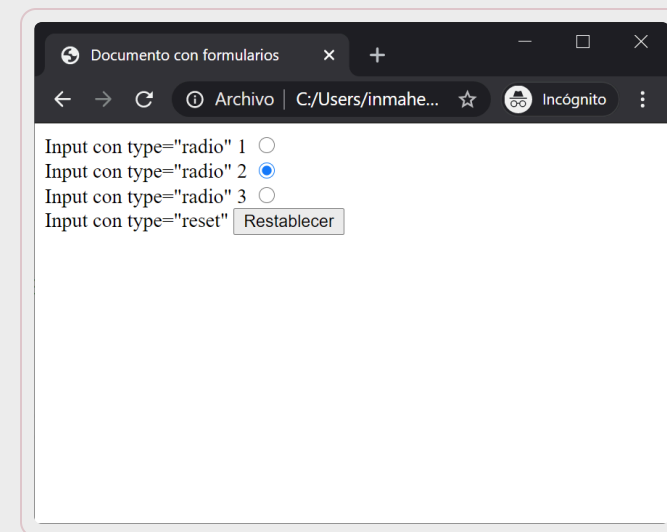
Radio buttons / Reseteo

```
<input ... type="radio"/>
```

- Genera un botón radio (1 de n). Todos los botones radio que tengan el mismo valor de name forman un grupo. Al servidor se le envía sólo el value del botón seleccionado.
- El atributo value no genera texto. El atributo checked hace que aparezca como seleccionado (sólo uno).

```
<input ... type="reset"/>
```

- Genera un botón para inicializar el formulario con los valores por defecto (los especificados en value).
- Para el texto del botón se puede usar value. Si no se especifica, los navegadores usan su propio texto por defecto. No necesita name.



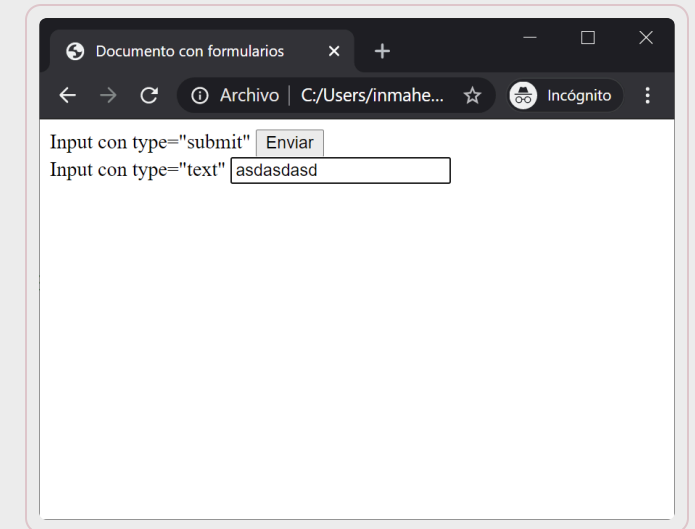
Submit / campos de texto

```
<input ... type="submit"/>
```

- Genera un botón para enviar los datos al servidor.
- Para el texto del botón se puede usar value o dejar al navegador decidir. No necesita name a menos que se quieran poner varios en un mismo formulario con distintos value y así saber cual se pulsó (Añadir/Modificar/Borrar).

```
<input ... type="text"/>
```

- Es el tipo por defecto. Genera un campo de texto de longitud máxima maxlength y de tamaño en pantalla size.
- Se puede mostrar el contenido esperado con el atributo placeholder
- Se puede usar value para darle un valor por defecto.



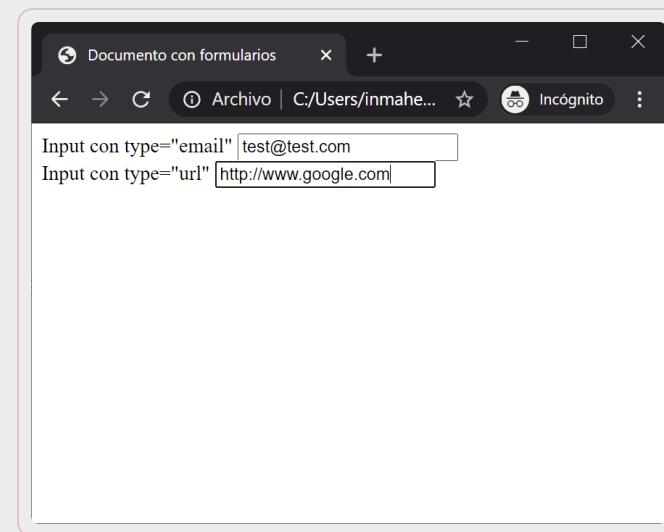
Email / url

```
<input ... type="email"/>
```

- Genera un campo de texto donde se espera una dirección de correo.
- Útil para que el navegador valide que el contenido es una dirección de correo bien formada. Algunos navegadores en dispositivos móviles ofrecerán un teclado adaptado al contenido esperado.

```
<input ... type="url"/>
```

- Genera un campo de texto donde se espera una url. (esquema://xxxx)
- Útil para que el navegador valide que el contenido es una url bien formada. Algunos navegadores en dispositivos móviles ofrecerán un teclado adaptado al contenido esperado.



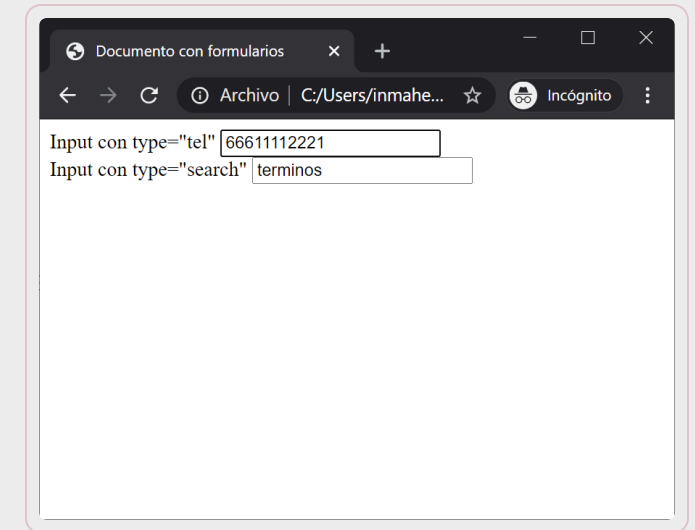
Campos de búsqueda / teléfonos

```
<input ... type="search" />
```

- Genera un campo de texto donde se espera palabras clave a buscar.
- Algunos navegadores renderizan el cuadro de una manera distinta (bordes redondeados) y aparece un icono para borrar el contenido.

```
<input ... type="tel" />
```

- Genera un campo de texto donde se espera un número de teléfono.
- Debido a la variedad de formatos para los números de teléfono, los navegadores no realizan ninguna validación. Algunos navegadores en dispositivos móviles pueden presentar un teclado específico.



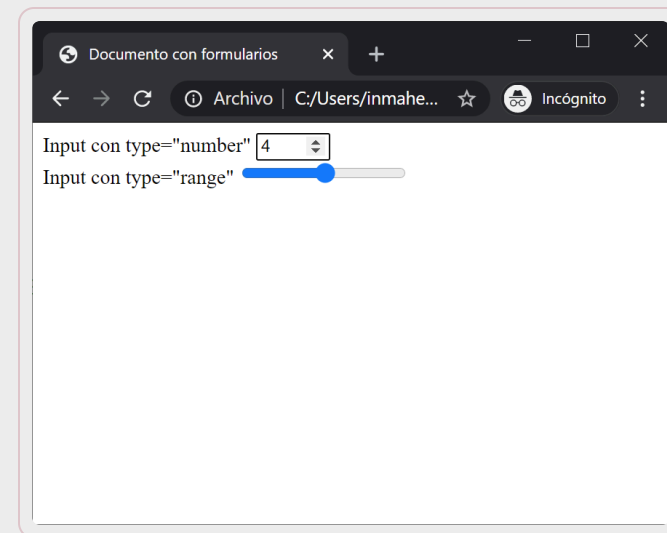
Números / Sliders

```
<input ... type="number"/>
```

- Genera un campo de texto donde se espera un valor numérico.
- Se pueden establecer condiciones a los valores aceptados:
 - min: valor mínimo aceptable
 - max: valor máximo aceptable
 - step: paso entre un valor aceptable y el siguiente
- Ejemplo, campo que recoge el peso de un paquete que se desea enviar (se supone que el peso máximo es 10kg y el sistema requiere conocerlo con precisión de un decimal).

```
<input ... type="range"/>
```

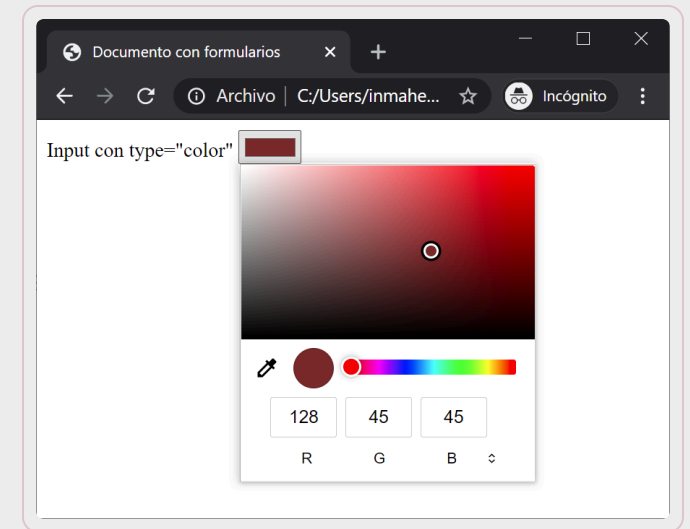
- Genera una barra de ajuste (slider).
- Acepta los mismos atributos que el input de tipo “number”.



Selector de colores

```
<input ... type="color"/>
```

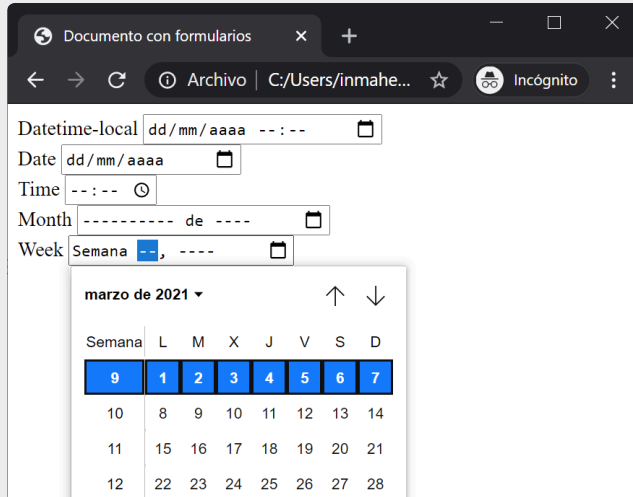
- Genera un selector de colores proporcionado por el navegador y/o el SO.
- El color seleccionado por el usuario se envía en formato hexadecimal.



Selector de fechas

```
<input ... type="datetime-local|date|month|week|time" />
```

- Generan un selector de fechas.
- Evitan el ingreso de fechas incorrectas.



The screenshot shows a web browser window with the title "Documento con formularios". The address bar shows "Archivo | C:/Users/inmahe...". The page contains a form with the following fields:

- Datetime-local: dd/mm/aaaa --:--
- Date: dd/mm/aaaa
- Time: --:--
- Month: ----- de ----
- Week: Semana --, ----

A calendar dropdown is open for the month of March 2021. The calendar shows a grid of days from 1 to 28. The days are arranged in a 4x7 grid. The first row shows the days 1 through 7. The second row shows 10 through 14. The third row shows 11 through 21. The fourth row shows 12 through 28. The days are labeled with their respective days of the week: L (Lunes), M (Martes), X (Miércoles), J (Jueves), V (Viernes), S (Sábado), and D (Domingo).

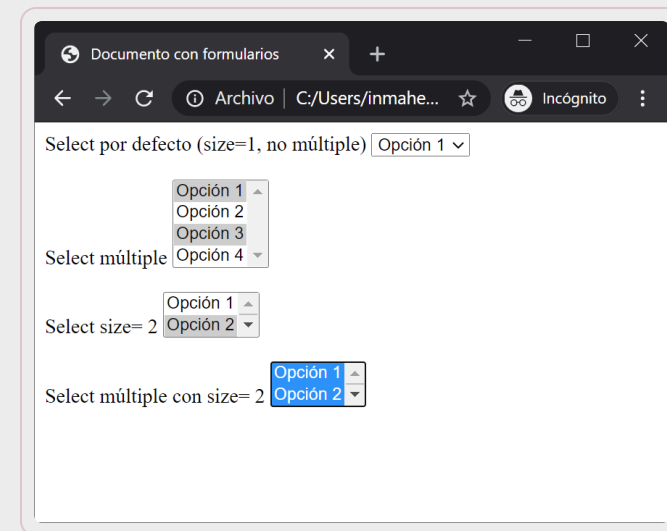
Elemento select

```
<select name="nombre para procesamiento en servidor"
        id="ident. unico para procesamiento en cliente"
        size="1..n"
        multiple
        disabled
        title="texto para el tooltip">
  <option selected>Valor1</option>
  <option value="value">Valor2</option>
  ...
  <option>Valorn</option>
</select>
```

Elemento de selección de valores (lista / combo box)

Alternativa a los radio si hay muchas opciones.

Los atributos `name`, `id`, `disabled` y `title` tiene el mismo significado que en `<input>`.



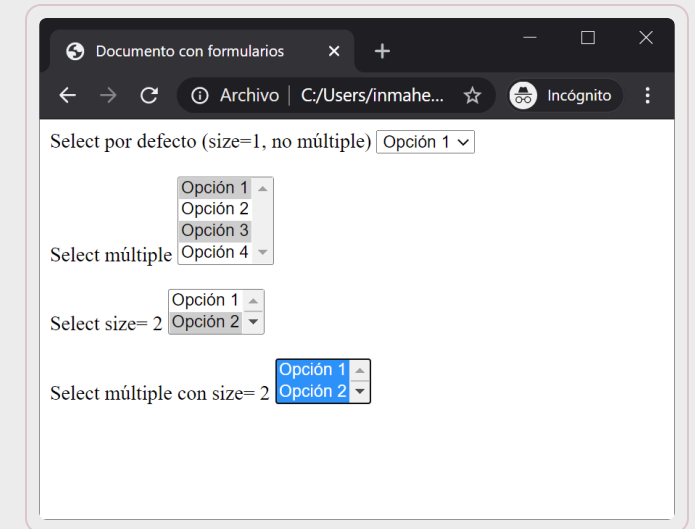
Atributos de `<select>`

multiple

- si está presente, indica que se permite seleccionar más de una opción, que llega al servidor como:
 - `name=value1&name=value2&...&name=valueN`

El atributo `multiple` hace que el control se visualice como una lista aunque `size` sea 1. Por defecto, la selección es simple.

`size` : indica el número de opciones visualizadas a la vez. Por defecto, el valor de `size` es 1. Si `size=1` y no es `multiple`, se visualiza como un combobox, si no, se visualiza como una lista de tamaño `size`.



Elemento `<option>`

Atributos de `<option>`

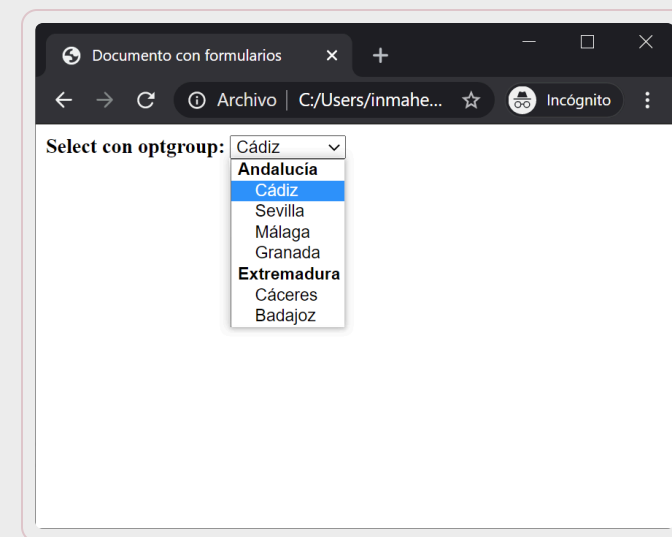
- `value` : si se especifica, es el valor que se envía al servidor, si no, se envía el contenido de la etiqueta.
- `selected` : indica las opciones seleccionadas por defecto. Si no se especifica, es la primera opción.

Elemento `<optgroup>`

- Permite agrupar opciones y darles un nombre mediante su atributo `label` .

Select con optgroup:

```
<select name="provincia">
  <optgroup label="Andalucía">
    <option>Cádiz</option>
    <option>Sevilla</option>
    <option>Málaga</option>
    <option>Granada</option>
  </optgroup>
  <optgroup label="Extremadura">
    <option>Cáceres</option>
    <option>Badajoz</option>
  </optgroup>
</select>
```



Elemento textarea

Permite que el usuario introduzca una o varias líneas de texto

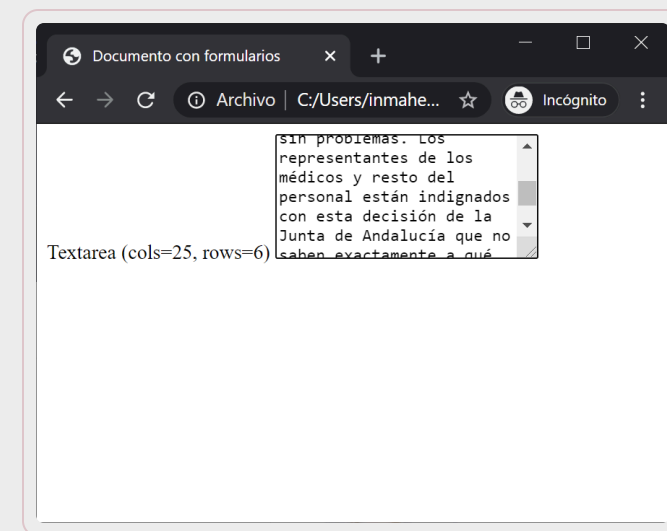
El contenido de la etiqueta se toma como valor por defecto.

```
<textarea name="nombre para procesamiento en servidor"
           id="ident. unico para procesamiento en cliente"
           cols="columnas de ancho" rows="filas de alto"
           disabled>
Texto por defecto.
</textarea>
```

Atributos de `<textarea>`

Los atributos `cols` y `rows` indican respectivamente el número de columnas y de filas del área de texto que se visualizará.

Los valores de `cols` y `rows` se interpretan como número de caracteres (aproximado).



Elemento `button`

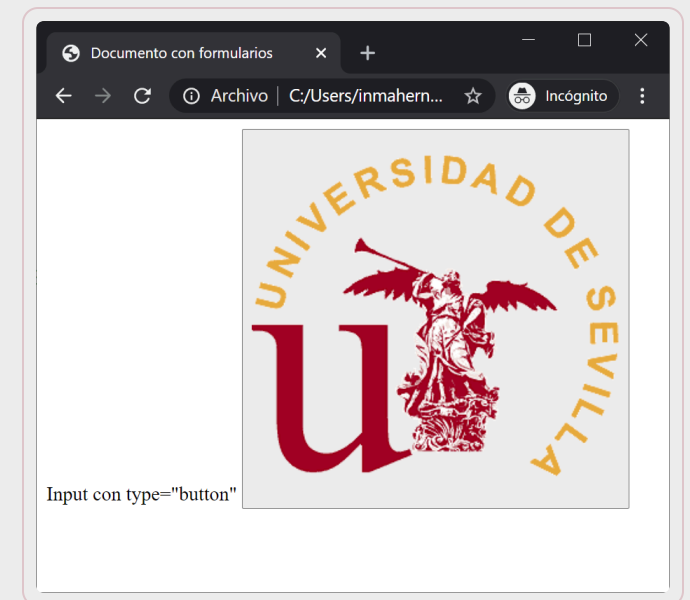
Permite definir botones con contenido complejo, no sólo texto, sino cualquier contenido HTML, incluyendo imágenes o texto con formato.

Similares a los botones de formulario, pero sin estar ligados a uno

```
<button name="nombre para procesamiento en servidor"
        id="ident. unico para procesamiento en servidor"
        value="valor para enviar al servidor"
        type="submit|reset|button"
        disabled>
  <!-- contenido del boton (cualquier elemento HTML) -->
</button>
```

Atributos de `<button>`

El atributo `type` indica el uso del botón, por defecto `button`, que equivale a `<input type="button">`.



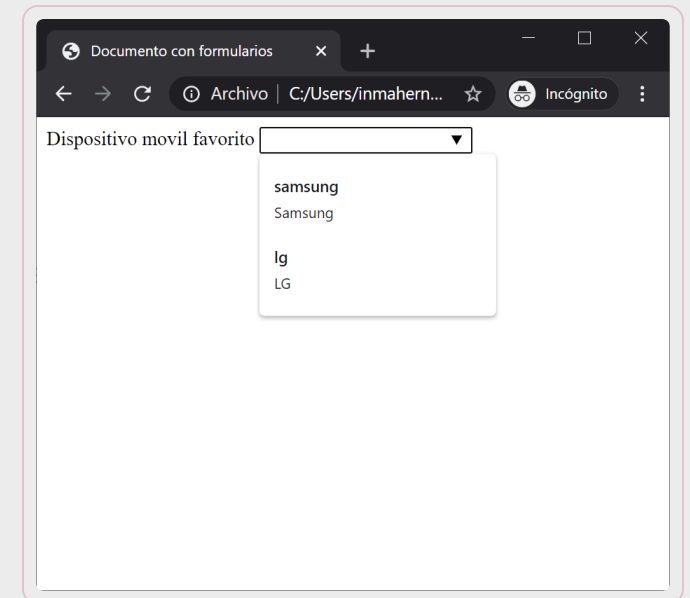
Elemento datalist

Permite generar sugerencias para un campo de texto ordinario.

```
<input id="movilFavorito" name="movilFavorito" type="text" list="opcionesMoviles">  
<datalist id="opcionesMoviles">  
  <option label="Samsung" value="samsung">  
  <option label="Apple" value="apple">  
  ...  
</datalist>
```

Atributos de `<datalist>`

El atributo `list` indica la lista de sugerencias asociada con este `input`

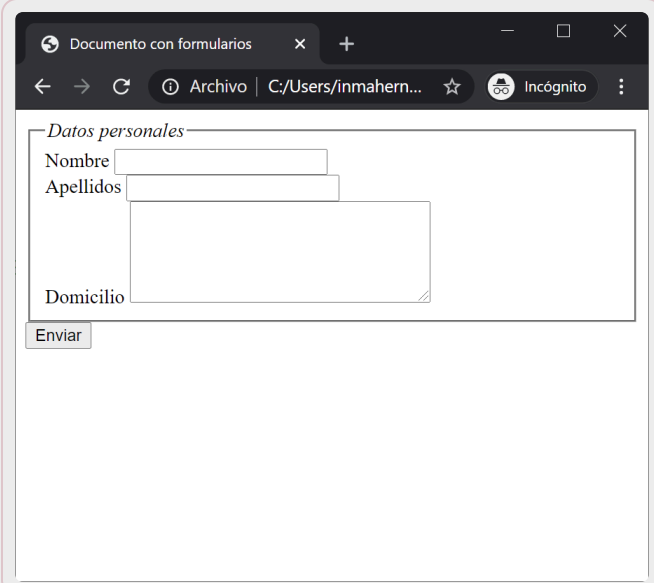


Elemento grupo de controles

```
<fieldset>
  <legend align="left|right">
    Título del grupo
  </legend>
  <!-- controles de formulario -->
</fieldset>
```

Crea un marco alrededor de un grupo de controles.

La etiqueta `legend` permite especificar el título del grupo con formato `HTML`.



The screenshot shows a web browser window with the title 'Documento con formularios'. The address bar shows 'Archivo | C:/Users/inmahern...'. The page content is a form titled 'Datos personales' which is enclosed in a box. Inside the box, there are three input fields: 'Nombre', 'Apellidos', and 'Domicilio'. Below these fields is a button labeled 'Enviar'.

Elemento de etiqueta de campo

```
<!-- uso 1 -->
<label for="id. de control">
  Texto etiqueta
</label>
<!-- control con id -->

<!-- uso 2 -->
<label>
  <!-- control -->
</label>
```

Permite indicar explícitamente la etiqueta o título de un control de un formulario.

Está pensado para herramientas de accesibilidad, que pueden leer en voz alta el contenido de una página `HTML` .

Se puede usar de dos formas, indicando la etiqueta y luego el control mediante el atributo `for` , o bien haciendo al control hijo de `<label>` .

No tiene una representación concreta, aunque se le pueden aplicar hojas de estilo.

Contenidos

Introducción

Elementos de formulario

Validación de formularios

Buenas prácticas



Validación de formularios

Es necesario validar los datos que el usuario ingresa en los formularios

La validación ha de realizarse en cliente y en servidor

En cliente se realiza la validación a través de las herramientas que integra `HTML` desde la versión 5 y/o con `JavaScript`.

- Con `HTML5` / `CSS` se pueden realizar las validaciones más comunes
 - Ejemplo: ¿Se ha dejado vacío un campo obligatorio? ¿Se ha introducido un email con formato incorrecto? ¿Se ha introducido un valor numérico inferior a un mínimo o superior a un máximo?
- Con `JavaScript` pueden además realizarse validaciones más complejas.
 - Ejemplo: ¿Es válido el DNI introducido? ¿Está ya en uso el email introducido?

Validación con HTML 5

Con `HTML5` se indica donde se desea realizar validación, pero no es necesario implementar todos los detalles.

Validación por tipo de campo:

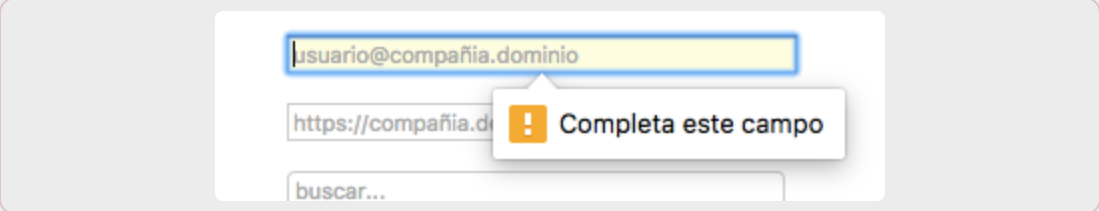
- Los `input` de tipo `email`, `url`, `number`, `range`, `date`, etc. realizan la validación básica. Ejemplos:
 - `email`: el contenido debe ser un conjunto de caracteres + `@` + otro conjunto de caracteres.
 - `url`: `schema://dominio`
 - `number`: sólo se admiten números
 - `date`: sólo se admiten fechas válidas



Campos obligatorios

Para marcar qué campos es establecen como obligatorios basta con incluir el atributo required en los campos obligatorios

```
<input .. required >
```



A screenshot of a web form with three input fields. The first field contains the text 'usuario@compañia.dominio' and has a yellow background. The second field contains 'https://compañia.d' and the third field contains 'buscar...'. A red error message box with a white exclamation mark icon is positioned over the first field, displaying the text 'Completa este campo'.

Expresiones regulares

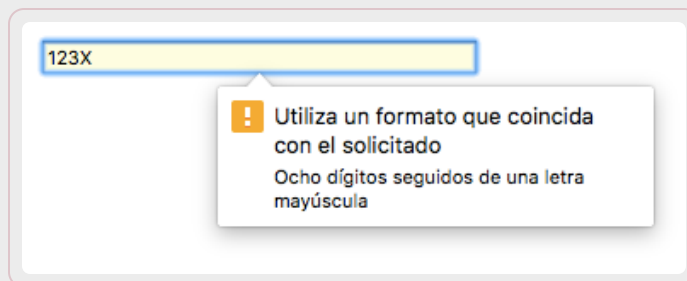
Es la herramienta más potente de validación en HTML5

En el atributo pattern se indica la expresión regular que deben cumplir los valores introducidos en un campo.

El contenido del atributo title es mostrado por el navegador si los datos introducidos no son válidos

Ejemplo: Validación simple de NIF:

```
<input type="text" placeholder="12345678X"  
pattern="^[0-9]{8}[A-Z]"  
title="Ocho dígitos seguidos de una letra mayúscula" >
```

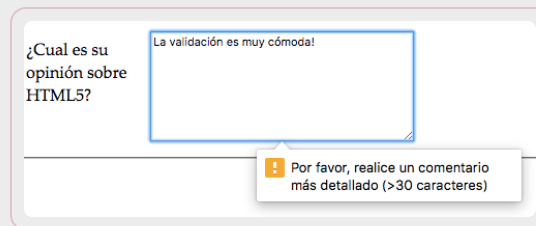


A screenshot of a web form with a text input field containing the placeholder text "123X". The field has a yellow background and a blue border. Below the field, a white tooltip with a yellow exclamation mark icon displays the error message: "Utiliza un formato que coincida con el solicitado" and "Ocho dígitos seguidos de una letra mayúscula".

Validación avanzada (HTML+JavaScript)

Para realizar validaciones más específicas de la aplicación, que no estén cubiertas por los mecanismos estándar de HTML5

```
<textarea id="comments" oninput="validarComentarios(this)">
</textarea>
...
<script>
function validarComentarios(input) {
  if (input.value.length <= 30) {
    input.setCustomValidity("Por favor, realice un comentario más detallado (>30 caracteres)");
  }
  else { // No hay error, limpiar los mensajes.
    input.setCustomValidity("");
  }
}
</script>
```



The screenshot shows a web form with the question "¿Cual es su opinión sobre HTML5?". There is a text input field. Above the field, a tooltip says "La validación es muy cómoda!". Below the field, a red error message is displayed: "Por favor, realice un comentario más detallado (>30 caracteres)".

¿Cómo implementar esta validación?

¿Se ha intentado registrar un usuario con un nombre de usuario que ya existe?

Nuevo usuario

! Ya existe una cuenta en idealista con ese email

Nuevo usuario

Agente

Solo puedes publicar en portales los anuncios que hayas creado o te hayan asignado

Nombre

Sandra

Apellidos

López

Email

slopez@idealista.com

Con el que accederá a idealista/tools

✉ Enviar invitación

Recibirá un email para crear el acceso

¿Y esta otra?

¿Se ha intentado cambiar la contraseña del usuario a una que ya usó en el pasado?

Cambio de Contraseñas Expiradas

Su contraseña de Intermático ha expirado.

Por su seguridad, las contraseñas expiran cada 365 días.

La nueva contraseña, debe ser diferente de las últimas 10 contraseñas que ha ingresado.

Sí desea cambiar su contraseña, ingrese la nueva contraseña y de clic en **Cambiar**.

Nueva Contraseña

Reingresar Contraseña

Conclusión

No siempre es posible realizar todas las validaciones en cliente y en servidor

- Hay validaciones que solamente pueden hacerse en servidor, dado que necesitan acceder a los datos de la BD

En general, siempre que sea posible, validar en cliente y en servidor:

- Validar en cliente muchas veces evita tener que hacer una petición “inútil” al servidor y gastar ancho de banda
- Es posible saltarse la validación en cliente (por ejemplo, usando una herramienta tipo REST Client), por lo que siempre es necesario validar también en servidor

Cuando no sea posible validar en cliente, hacerlo en servidor, pero preparar el cliente para mostrar los mensajes de error adecuados en caso de que la validación falle

Contenidos

Introducción

Elementos de formulario

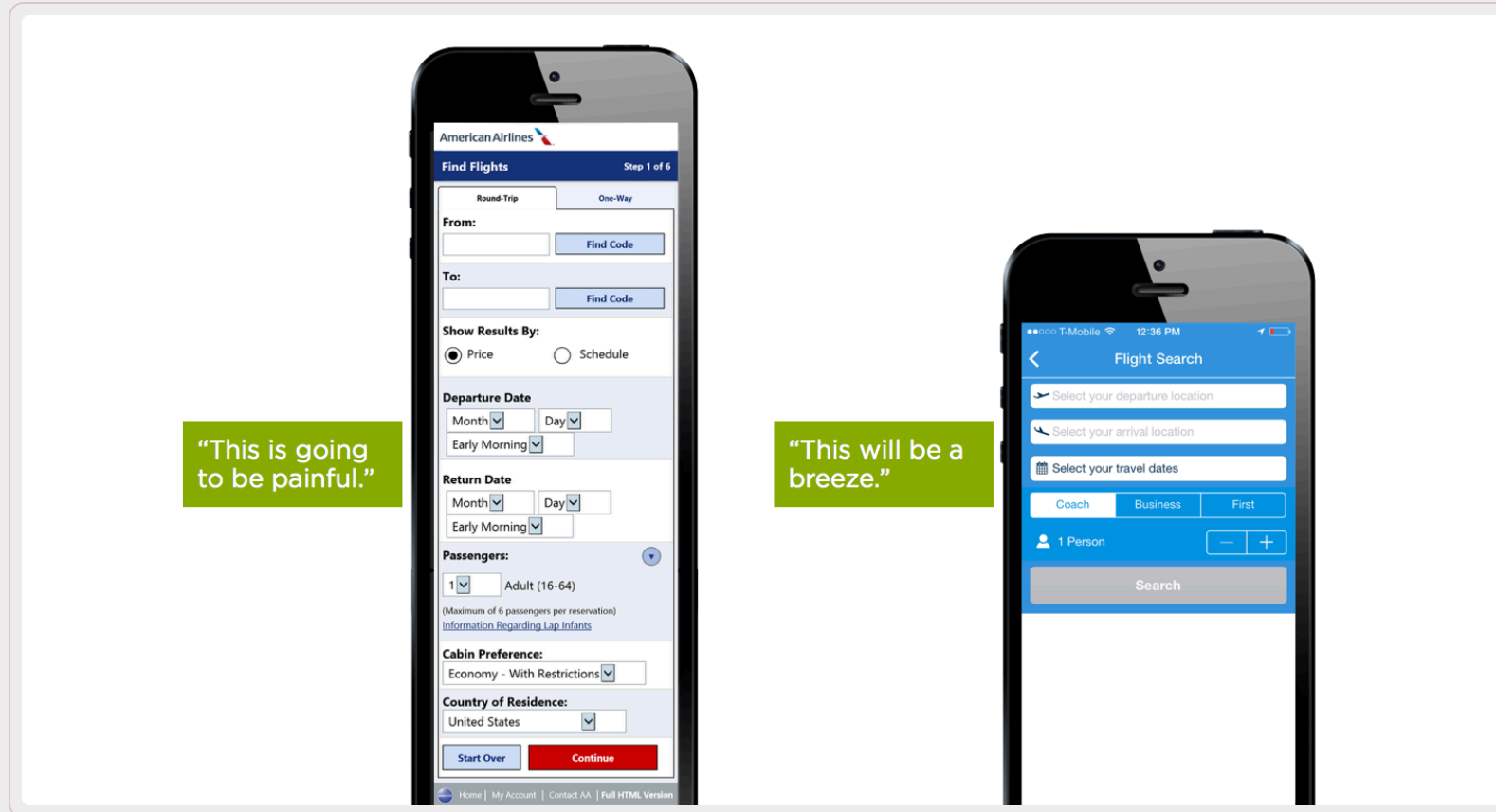
Validación de formularios

Buenas prácticas



Buenas prácticas – diseño de formularios

No solicitar información que no se vaya a usar



Buenas prácticas – diseño de formularios

Reducir el esfuerzo del usuario (minimizar campos de texto, usar más radios, checkboxes, combos)

The diagram illustrates a comparison between two form designs for the question "Where did you hear of us?". On the left, a light blue box contains the question above a single, wide text input field. On the right, a light blue box contains the same question above a vertical list of five options, each preceded by an unchecked checkbox. A blue arrow points from the text field on the left towards the checkbox list on the right, indicating a transition or preference for the more structured option.

Where did you hear of us?

☐ Television Ad

☐ Newspaper Ad

☐ Website Ad

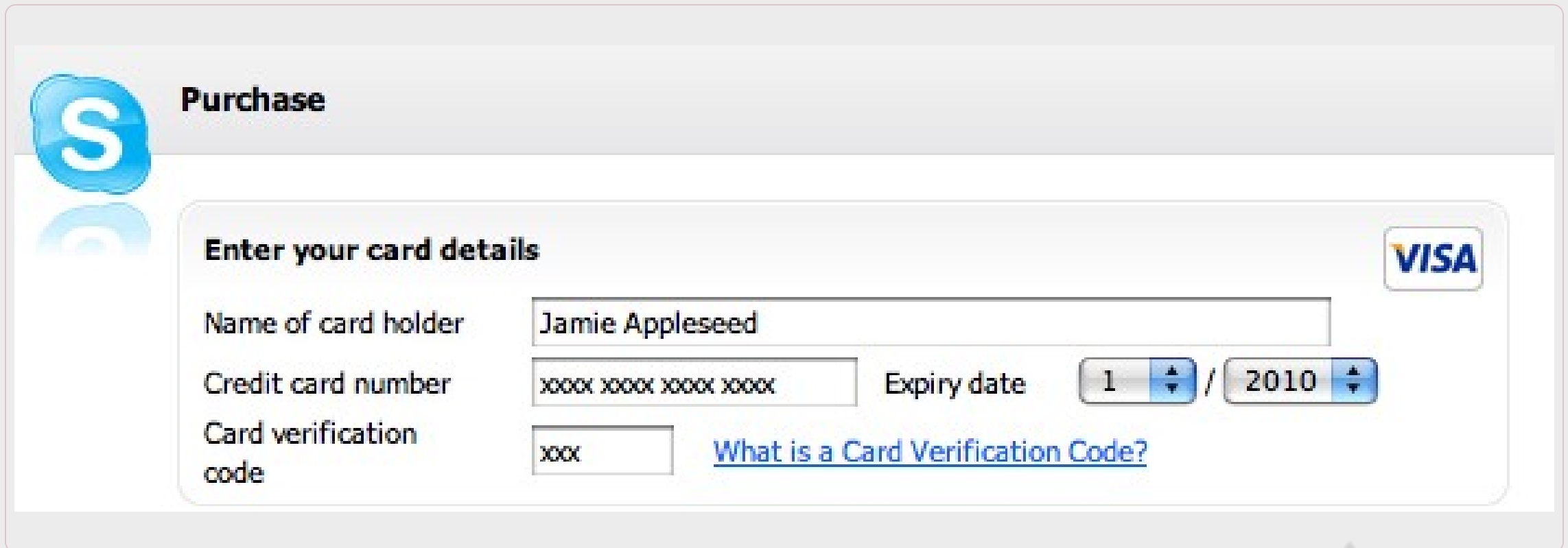
☐ From a Friend

☐ Some other place

En los campos de texto, ser flexible con el formato (ej: admitir telefonos con/sin guiones)

Buenas prácticas – diseño de formularios

Ajustar el tamaño de los campos de texto a la entrada esperada (ej: un C.P., 5 caracteres, un nombre ~ 20 caracteres)



Purchase

Enter your card details 

Name of card holder

Credit card number Expiry date /

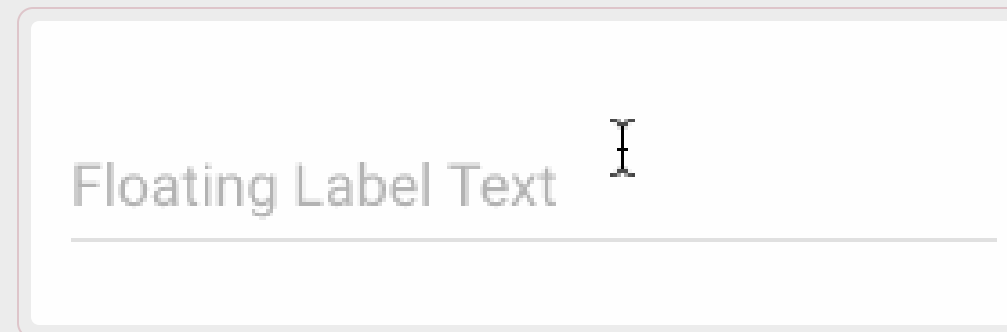
Card verification code [What is a Card Verification Code?](#)

Buenas prácticas – diseño de formularios

No usar los placeholders como etiquetas de los campos de texto (cuando el usuario empieza a escribir, los placeholders desaparecen)



Alternativa: etiqueta flotante



Buenas prácticas – diseño de formularios

Distinguir campos obligatorios y opcionales

Billing Address
☐ Same as Delivery/Shipping Address
First Name

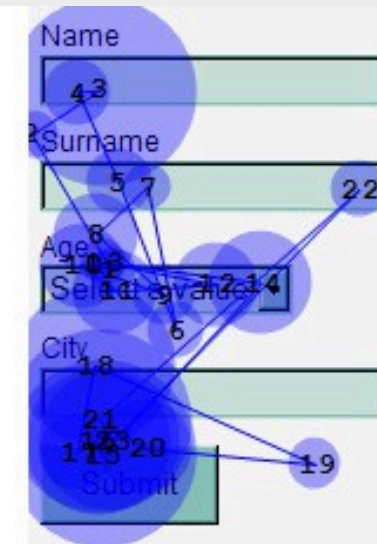
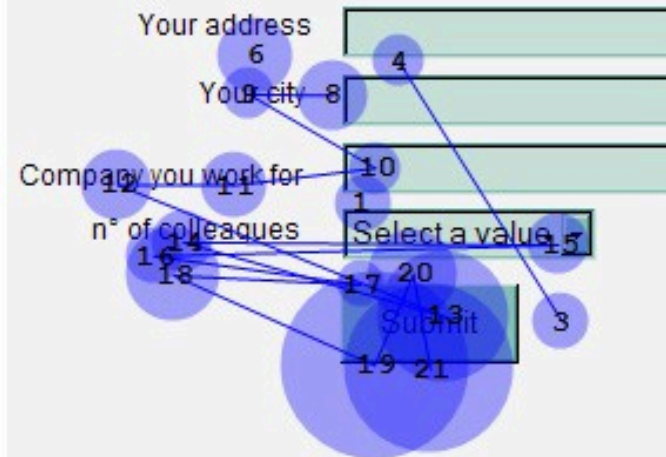
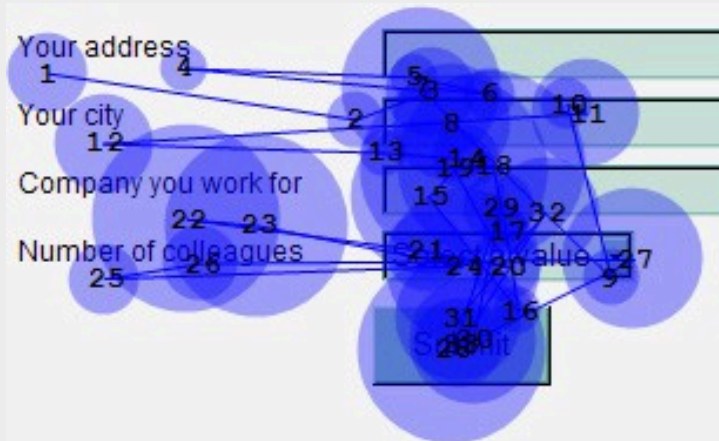
Last Name

Street Address Line 1

Street Address Line 2 Optional

Buenas prácticas – diseño de formularios

Poner las etiquetas encima de los campos (hace más rápida la lectura)



Buenas prácticas – diseño de formularios

Proporcionar al usuario mensajes de error claros, bien visibles y que impliquen usabilidad:

- Mostrar el error cuando el usuario introduce el dato, no esperar a que el usuario envíe el formulario (si es posible)
- Usar un formato distintivo (colores, tamaños)
- Mostrar los errores preferentemente encima del formulario o cerca en el primer campo erróneo (se ven mejor que debajo)
- Si los campos requieren un formato específico, avisarlo de antemano
- Cuando el usuario introduzca un dato erróneo, no vaciar los datos correctos previamente introducidos.

Ejercicio propuesto

Abrir un editor de código

Escribir un documento html que contenga un formulario con un campo de texto y un botón para enviar

Probar con distintos tipos de campos de entrada: Selectores de fecha, colores, combo boxes, etc.



Conclusiones

Los **formularios HTML5** permiten recoger datos del usuario y enviarlos al servidor mediante elementos como **form**, **input**, **select**, **textarea** y **button**.

Cada control tiene atributos específicos que definen su **comportamiento**, su **valor enviado** y su interacción con el usuario.

Elegir bien el tipo de control mejora la **usabilidad**, reduce errores y facilita la introducción de datos.

La semántica de los elementos de formulario también influye en la **accesibilidad** y en la claridad de la interfaz.

Conclusiones

La **validación** debe hacerse tanto en **cliente** como en **servidor**, porque no todos los errores pueden detectarse en el navegador ni toda validación cliente es fiable por sí sola.

HTML5 incorpora mecanismos de validación básicos mediante **tipos de input, required y pattern**, que pueden complementarse con **JavaScript** para reglas más complejas.

Un buen formulario debe aplicar **buenas prácticas** de diseño: pedir solo lo necesario, minimizar esfuerzo, mostrar errores claros y distinguir bien los campos obligatorios.

Diseñar formularios correctos no es solo una cuestión técnica, sino también de **experiencia de usuario y eficiencia** en la interacción.



Gracias

Universidad de Sevilla

**Departamento de Lenguajes y Sistemas
Informáticos**

UNIVERSIDAD DE SEVILLA